Programming with Generative AI

Viraj Kumar
Kotak-IISc AI-ML Centre

Lecture 3:

- Decomposing complex tasks into simpler tasks
- Variables and Assignments



A CSR Initiative by



NPTEL

Complex Expressions

- How would you calculate this using a simple calculator?

```
>>> (11 + 19) / (19 + 31)
0.6
```

We have **decomposed** a complex task...

...into a sequence of several simpler sub-tasks

assignment statements

```
>>> M = 19 + 31
>>> 11 + 19
>>> M
0.6
```

expression with **variables**

```
>>> numerator = 11 + 19
>>> denominator = 19 + 31
>>> numerator / denominator
0.6
```

A Python **program**

Assignment Statements

- General form:

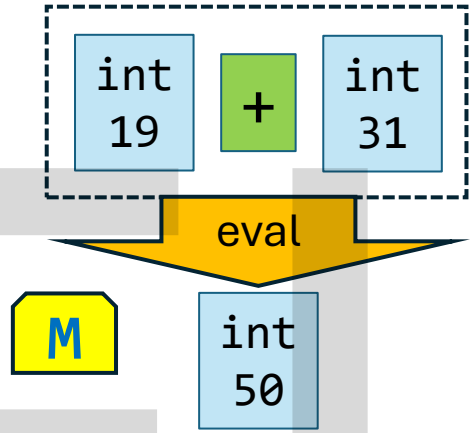
  `variable = expression`

- This ***always*** performs two steps:
  1. Evaluate ***expression*** as an object (replace each variable in expression with the object it *currently* labels)
  2. Label this object with the name `variable`

     (Do ***not*** modify any labels that the evaluated object already has)

Example 1:

```
M = 19 + 31
```



NPTEL

Assignment Statements

- General form:

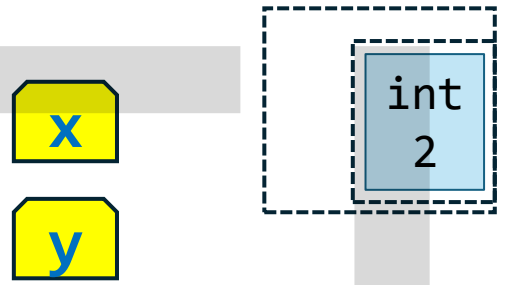
  `variable = expression`

- This ***always*** performs two steps:
  1. Evaluate ***expression*** as an object (replace each variable in expression with the object it *currently* labels)
  2. Label this object with the name **`variable`**

     (Do ***not*** modify any labels that the evaluated object already has)

Example 2:

```
x = 2
y = x
```



NPTEL

Assignment Statements

- General form:

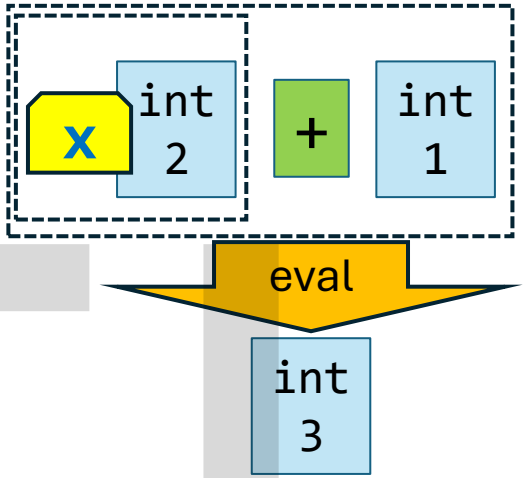
  `variable = expression`

- This ***always*** performs two steps:
  1. Evaluate ***expression*** as an object (replace each variable in expression with the object it *currently* labels)
  2. Label this object with the name **`variable`**

     (Do ***not*** modify any labels that the evaluated object already has)

Example 3:

```
x = 2
x = x + 1
```



NPTEL

Test Our Friend's Understanding

```
>>>type(7)
<class 'int'>
>>>7 + 1
8
```

```
>>>x = 7
>>>type(x)
<class 'int'>
>>>x + 1
8
```

```
>>>x = '1947'
>>>'August' + x
```



A **variable** is an **object**: it has a **type** and a **value**

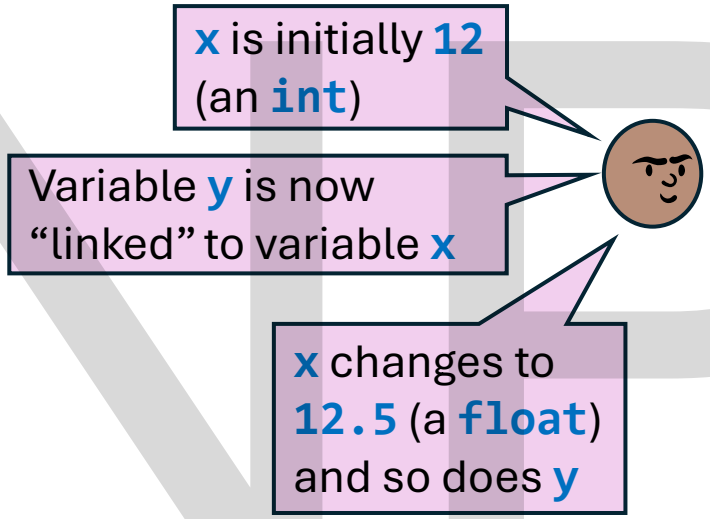
"The value/type of the assigned to is ..."

```
TypeError: can only concatenate str (not "int")
to str
```

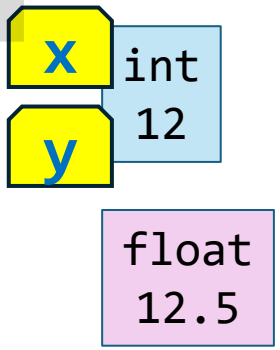
dynamically typed

NPTEL

Test Our Friend's Understanding




```
>>> x = 25 // 2
>>> y = x
>>> x = 25 / 2
>>> y
```



Test Our Friend's Understanding

A variable is like a box that holds a value, and y = x copies data from box x into box y



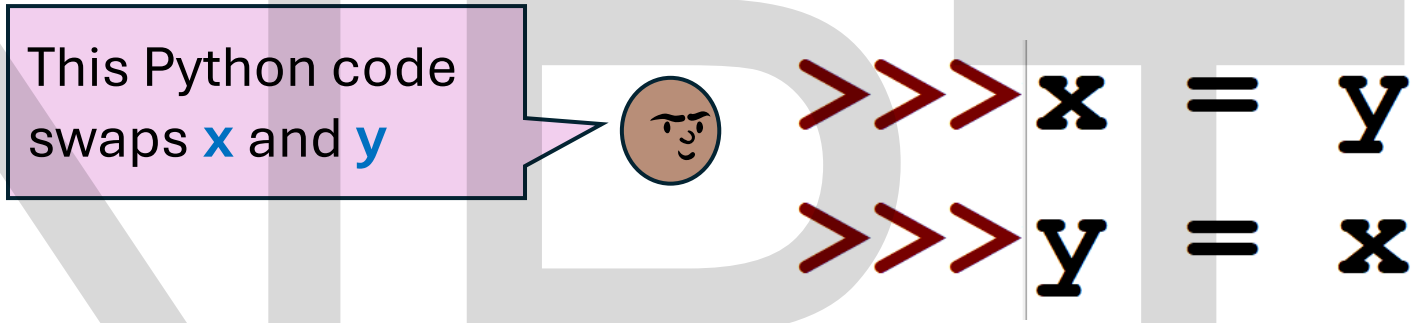
```
>>> x = 25 // 2
>>> y = x
>>> x = 25 / 2
>>> y
12
```

PythonTutor: **incorrect** visualization (*default*)

PythonTutor: **correct** visualization

NPTEL

Test Our Friend's Understanding



- Let's visualize this on PythonTutor
  - Note the use of **comments**!
- Correct ways to swap:
  - With a temporary variable
  - "Without" a temporary variable

NPTEL

Always choose meaningful variable names

Good names

- `twice_n`
- `total`
- `return_value`
- `temperature`

Bad/illegal names

- `n*2` (or: `2_n`)
- `sum`
- `return` (or: `RetVal`)

Avoid **upper-case** letters, use _ to separate words

You know what `t` means today! **Others** who read your code may not understand what `t` means.
Others includes:

- Your colleagues
- You (in the future)
- AI

NPTEL

Summary

- Assignments help decompose complex tasks into steps

  `var = expr`

  - Variables hold intermediate values
  - = vs. ==
- A Python variable is merely a ***label***
  - Evaluate ***expr***, label object with `var`
  - If `var` in ***expr***, use *current* value of its object
- Help **others** understand your code
  - Use meaningful variable names, comments

NPTEL